

MemoryOS for AI Agents

Self-Healing Memory Architecture with Real-Time Drift Detection

B.Tech CSE-AI Major Project Report | Parul University | 2025-26
Aaditya Rana (2303051240001) & Shivam Kumar Poddar (2303051240374)
Supervisor: Ms. Kiran Sharma

Abstract

MemoryOS is an open-source, operating-system-inspired memory management layer for LLM-based AI agents. It addresses the critical problem of *Context Rot* — the silent degradation of agent memory that causes 65% of production AI failures. MemoryOS provides real-time drift detection using cosine similarity, zero-downtime auto-healing via relevance decay pruning, and multi-agent knowledge synchronization. Benchmark results demonstrate **90.9% drift reduction** and **64.7% consistency improvement** over agents without memory management. All code is open-sourced at github.com/shivampoddar12/memoryos-project.

1. Problem Statement

Statistic	Value	Source
Agent failures due to memory loss	65%	Industry Research 2026
Market problem by 2031	\$800 Billion	Gartner, 2026
Pilots never reaching production	75%	McKinsey, 2026
Open-source memory health tools	0	This Work

AI agents deployed in production exhibit a consistent failure pattern: reliable performance during evaluation, followed by silent quality decay after sustained use. This phenomenon — termed *Context Rot* — occurs when critical earlier context is displaced by newer tokens. No existing open-source tool monitors or heals agent memory health.

2. Methodology

2.1 Drift Detection Algorithm

Each agent session is embedded using TF-IDF (500 features, bigram support). A baseline vector is captured at deployment time. Per-session drift is computed as:

```
drift(t) = 1 - cosine_similarity(v_baseline, v_current)
If drift(t) >= 0 (0.45):
    trigger Auto-Heal
```

2.2 Auto-Heal Module

On threshold breach, the Auto-Heal Module executes a three-step transaction without restarting the agent: (1) compute relevance decay for all memory items using $rel(m,t) = importance(m) \times e^{(-\lambda \cdot \Delta t)}$, (2) prune items below relevance threshold 0.3, (3) re-inject top-3 highest-importance memories into working context.

2.3 Threshold Calibration

The drift threshold θ was calibrated empirically across 11 candidate values (0.20 to 0.70) using 4 scenario types: similar, borderline, different, and extreme. $\theta = 0.45$ was identified as optimal, correctly classifying all

4 scenario types.

3. Results

Metric	WITHOUT MemoryOS	WITH MemoryOS	Improvement
Average Drift Score	0.7113400000000001	0.06448000000000001	90.9% reduction
Avg Consistency	28.87%	93.55%	+64.7%
Heal Events	0	8	Auto-triggered
Sessions Tested	10	10	10 long-horizon

4. Technology Stack

Layer	Technology	Purpose
Agent Framework	LangGraph 1.2+	Stateful agent graph orchestration
Embeddings	TF-IDF (sklearn)	500-dim text vectorization + bigrams
Drift Detection	NumPy / SciPy	Cosine similarity computation
Dashboard	Streamlit + Plotly	Real-time health visualization
Multi-Agent	Shared Bus (custom)	Knowledge synchronization
Version Control	Git + GitHub	Reproducible experiments

5. References

[1] C. Packer et al., "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560, 2023.

[2] P. Chhikara et al., "Mem0: Building Production-Ready AI Agents," arXiv:2504.19413, ECAI 2025.

[3] A. Rath, "Agent Drift: Quantifying Behavioral Degradation," arXiv:2601.04170, 2026.

[4] P. Rasmussen et al., "Zep: A Temporal Knowledge Graph for Agent Memory," arXiv:2501.13956, 2025.

[5] M. Kang et al., "ACON: Context Compression for Long-Horizon Agents," arXiv:2510.00615, 2026.

[6] Y. Hu et al., "MemoryAgentBench: Evaluation Framework," arXiv:2507.05257, 2025.